

MPhil DIS C2 Coursework Report

Cholesky Factorization on CSD3
University of Cambridge

Candidate Name: Chang Xu
Raven ID: cx272

March 31, 2026

Contents

1	Introduction	3
2	Method and version evolution	4
2.1	Baseline algorithm and data layout	4
2.2	Tag table and optimisation method	4
2.2.1	v0: baseline serial reference	4
2.2.2	v0 to v1: serial kernel clean-up	5
2.2.3	v1 to v2: blocked reformulation	5
2.2.4	v2 to v3: first OpenMP parallelisation	6
2.2.5	v3 to v4: final parallel tuning	6
3	Testing and experimental setup	8
3.1	Experimental setup and reproducibility	8
3.1.1	Platform, build, and benchmark configuration	8
3.1.2	Benchmark inputs and timing protocol	8
3.1.3	Benchmark coverage	8
3.1.4	Benchmark matrix generation	9
3.1.5	Scaling metrics	9
3.1.6	Reproducibility	9
3.2	Test suite and test setup	9
3.2.1	Functional correctness	10
3.2.2	Regression against trusted and external references	10
3.2.3	Stability under block-size changes	10
3.2.4	OpenMP consistency	11
4	Performance results	12
4.1	Validation summary	12
4.2	Performance across versions	12
4.2.1	v0 to v1: serial kernel clean-up	13
4.2.2	v1 to v2: blocking for locality	15

4.2.3	v2 to v3: OpenMP on the trailing update	15
4.2.4	v3 to v4: refined parallel kernel	16
4.3	Effect of OpenMP	17
4.3.1	Scaling with thread count at a fixed problem size	17
4.3.2	Dependence of thread benefit on matrix size	18
4.3.3	Schedule sensitivity	19
4.3.4	Summary	20
5	Discussion and conclusion	21
A	Appendix	22
.1	Full validation results	22
.2	Appendix: Canonical benchmark result tables	23
.2.1	Benchmark bundle overview	23
.2.2	Validation summary	23
.2.3	Full sweep tables	24
.2.4	OpenMP thread scaling	26
.2.5	Schedule study at $n = 1024$, 4 threads	26
.2.6	Reference log-determinant checks	27
.2.7	Representative hardware-counter summaries	27
.2.8	Extended thread-scaling derived metrics	28
.3	Code evolution across milestone versions	30
.3.1	v0: Baseline serial kernel	30
.3.2	v1: Serial kernel clean-up	31
.3.3	v2: Blocked serial formulation	32
.3.4	v3: First OpenMP version	33
.3.5	v4: Final tuned OpenMP kernel	35

Introduction

Cholesky factorization decomposes a symmetric positive-definite matrix C as $C = LL^T$, where L is lower triangular [1]. This coursework implements a in-place Cholesky routine in C++ and improve its performance on a single CSD3 CPU node using OpenMP.

The implementation was developed through tagged versions, from a correct serial baseline to optimised serial and parallel kernels. The main performance gains for larger matrices came from OpenMP parallelisation of the blocked trailing update, with blocking improving locality and final kernel tuning further improving parallel efficiency and thread-level scaling.

Method and version evolution

2.1 Baseline algorithm and data layout

The implementation computes an in-place Cholesky factorization of a symmetric positive-definite matrix $C \in \mathbb{R}^{n \times n}$, stored as a one-dimensional row-major `double` array [1]. After factorization, the lower triangle stores L and the upper triangle stores mirrored entries of L^T , matching the coursework interface. All versions retain the $\mathcal{O}(n^3)$ arithmetic cost of dense Cholesky, while blocking and OpenMP change locality and parallel execution quality.

2.2 Tag table and optimisation method

Tag name	Commit SHA	Milestone meaning	Used in report
v0-baseline	87423be	First correct serial baseline implementation	§2.2.1, Fig. 4.1–4.4
v1-serial-opt	0fd83ec	Serial kernel clean-up with row pointers and <code>inv_diag</code>	§2.2.2, Fig. 4.1–4.5
v2-blocked	c0e78a5	First working blocked serial formulation	§2.2.3, Fig. 4.1, Fig. 4.6
v3-openmp	f164e8e	First working OpenMP version on trailing update	§2.2.4, Fig. 4.1, Fig. 4.7
v4-final	533f427	Final tuned OpenMP version used for canonical results	§2.2.5, Fig. 4.1, Fig. 4.8–4.11

Table 2.1: Tagged commits used as milestone versions in this report.

Table 2.1 maps each milestone version used in this report to the corresponding repository tag and commit (Commit SHAs are short hashes from the repository tags), so that the optimisation path and benchmark bundles can be verified directly. Appendix A3 presents the Cholesky implementation for each milestone version.

2.2.1 v0: baseline serial reference

Version v0 was implemented as a direct serial Cholesky factorisation following the coursework baseline. For each pivot index p , the code computes the diagonal element

$$L_{pp} = \sqrt{A_{pp}}, \quad (2.1)$$

then scales the off-diagonal entries,

$$L_{ip} = \frac{A_{ip}}{L_{pp}}, \quad i > p, \quad (2.2)$$

and updates the trailing submatrix,

$$A_{ij} \leftarrow A_{ij} - L_{ip}L_{jp}, \quad i, j > p. \quad (2.3)$$

The implementation therefore consists of a pivot loop, an off-diagonal normalisation loop, and a doubly nested trailing-update loop. Its arithmetic cost is the expected $\mathcal{O}(n^3)$ [3], but it is the least cache-friendly version because the trailing matrix is accessed in an unblocked way, leading to high loop overhead.

2.2.2 v0 to v1: serial kernel clean-up

The first optimisation step preserved the mathematical algorithm but improved the inner serial kernel to reduce overhead in the most frequently executed loops.

The main changes were:

- reuse of row pointers, replacing repeated address calculations such as `c[i*n+j]` with direct row access;
- replacement of repeated division by multiplication with a precomputed reciprocal,

$$\text{inv_diag} = \frac{1}{L_{pp}}, \quad (2.4)$$

so that

$$\frac{A_{ip}}{L_{pp}} = A_{ip} \cdot \text{inv_diag}; \quad (2.5)$$

- reorganisation of the trailing-update loop to reuse pivot-row values and make memory access more regular.

The target was less repeated index computation at instruction-level efficiency, a cleaner loop body for the compiler, and a more regular update access pattern, with the aim of improving single-core throughput.

2.2.3 v1 to v2: blocked reformulation

Version v2 use blocking instead of processing one pivot at a time over the full remaining matrix, the matrix is partitioned into panels and trailing tiles of width b , where b is the block size [2] [3].

For a panel beginning at index k , the matrix is written as

$$A = \begin{pmatrix} A_{kk} & A_{k2}^T \\ A_{k2} & A_{22} \end{pmatrix}, \quad (2.6)$$

where A_{kk} is the current diagonal block. The blocked Cholesky step then has three stages.

1. Diagonal-panel factorisation. The current diagonal block is factorised as

$$A_{kk} = L_{kk} L_{kk}^T, \quad (2.7)$$

using the same unblocked inner kernel over the panel range `block_start: block_end`.

2. Block-column solve. The block below the panel is updated by solving

$$L_{ik} = A_{ik} L_{kk}^{-T}, \quad i > k. \quad (2.8)$$

In the code, this is implemented as a loop over rows below the panel and columns within the panel, with each entry formed from a short dot product followed by division by the panel diagonal.

3. Trailing-matrix update. The remaining matrix is updated as

$$A_{22} \leftarrow A_{22} - L_{2k} L_{2k}^T, \quad (2.9)$$

and in the implementation this is applied tile by tile:

$$A_{ij} \leftarrow A_{ij} - \sum_{p=k}^{k+b-1} L_{ip} L_{jp}. \quad (2.10)$$

The optimisation target was improved cache locality, especially in the trailing update. Compared with the unblocked algorithm, the blocked version reuses panel data and the current trailing tile over a smaller working region, reducing cache pressure and improving temporal locality.

2.2.4 v2 to v3: first OpenMP parallelisation

Once the blocked serial structure was in place, the trailing-matrix update was chosen as the main parallel region, because for a fixed panel its tiles are independent.

After panel factorisation and the block-column solve, the trailing update has the form

$$A_{ij} \leftarrow A_{ij} - \sum_{p=k}^{k+b-1} L_{ip} L_{jp}, \quad (2.11)$$

for all tiles in the remaining lower-triangular region.

This is implemented by placing an OpenMP `parallel for` over the outer tile loop:

```
#pragma omp parallel for schedule(static)
for (int i_block = block_end; i_block < n; i_block += block_size) {
    ...
}
```

The panel factorisation was not parallelised first because it is smaller, more sequential, and less important to total runtime, whereas the trailing update dominates the floating-point work for large matrices.

2.2.5 v3 to v4: final parallel tuning

The final version retains the blocked OpenMP structure while refining both parallel execution and inner-kernel efficiency[5].

First, scheduling is made runtime-configurable through

```
#pragma omp parallel for schedule(runtime)
```

so that `static`, `dynamic`, and `guided` policies can be compared under environment control. This is motivated by the irregular triangular workload of the blocked trailing update.

Second, the block-column solve is parallelised when the remaining problem is large enough. This stage computes

$$L_{ij} = \frac{1}{L_{jj}} \left(A_{ij} - \sum_{p=k}^{j-1} L_{ip} L_{jp} \right), \quad i \geq \text{block_end}, j \in [k, k+b), \quad (2.12)$$

so that, once the current panel has been factorised, rows below it can be distributed across threads.

Third, `omp simd` reductions are added to the inner dot-product kernels,

$$\sum_p L_{ip} L_{jp}, \quad (2.13)$$

using

```
#pragma omp simd reduction(+:dot)
```

to encourage compiler vectorisation.

Overall, v4 improves load balance in the trailing update, reduces block-column overhead, and increases SIMD utilisation in the inner kernels.

Testing and experimental setup

3.1 Experimental setup and reproducibility

3.1.1 Platform, build, and benchmark configuration

All canonical performance results in this report were obtained under a fixed execution configuration. The purpose of reporting these settings explicitly is reproducibility: the reported timings should be read together with the platform, build, and runtime metadata used to produce them.

Item	Setting
CSD3 partition	icelake
Node	cpu-q-66
Compiler	GNU 8.5.0
Build type	Release
Compilation flags	-O3 -DNDEBUG -std=c++17 -Wall -Wextra -Wpedantic -fopenmp
OpenMP placement	OMP_PLACES=cores
OpenMP binding	OMP_PROC_BIND=close
Thread count	4
Schedule policy / chunk	guided / 1
Block size	32
Warm-up runs	1
Measured repetitions	7
Reported timing statistic	median runtime

Table 3.1: Canonical benchmark configuration used for the main reported performance results.

Table 3.1 reports the canonical 4-thread configuration. The extended thread-scaling study in §4.3 used a wider sweep on icelake, where `lscpu` reported 76 CPUs as 2 sockets $\times$ 38 cores with 1 thread per core. Thread counts up to 64 still use physical cores only and do not enter an SMT or hyperthreading regime.

3.1.2 Benchmark inputs and timing protocol

Benchmark matrices were generated using the correlation-based construction from the coursework specification, with the diagonal set to unity to ensure a symmetric positive-definite input.

For each case, factorisation was run on a fresh copy of the matrix, and runtime was measured as own wall-clock time. Each configuration used one warm-up run and seven measured repetitions with the median runtime reported as the main timing statistic to reduce sensitivity to transient system noise. Performance is also reported in GFLOP/s using the standard dense Cholesky operation count.

3.1.3 Benchmark coverage

The benchmark suite evaluated:

- correctness against reference results;
- runtime scaling across matrix sizes;
- thread scaling of the OpenMP versions;
- schedule sensitivity of the final tuned implementation;
- stability across repeated runs and block sizes.

3.1.4 Benchmark matrix generation

Large benchmark matrices were generated from the coursework correlation kernel,

$$C_{ij} = 0.99 \exp\left(-\frac{1}{2} \cdot 16 \cdot \frac{(i-j)^2}{n^2}\right), \quad C_{ii} = 1, \quad (3.1)$$

for $0 \leq i, j < n$.

3.1.5 Scaling metrics

Speedup is defined as

$$S(p) = \frac{T_1}{T_p}, \quad (3.2)$$

where T_p is the median runtime measured with p threads. Efficiency is

$$E(p) = \frac{S(p)}{p}. \quad (3.3)$$

The Karp–Flatt serial fraction is

$$e(p) = \frac{1/S(p) - 1/p}{1 - 1/p}, \quad (3.4)$$

which estimates the apparent non-parallel component, including both true serial work and parallel overhead. All three metrics are computed from median runtimes over repeated runs.

3.1.6 Reproducibility

Each major optimisation stage was associated with a tagged code version, and the reported comparisons are made between these milestone implementations. **Table 3.1** lists the platform, build, and runtime metadata used for the canonical benchmark runs so that the reported measurements can be reproduced consistently.

3.2 Test suite and test setup

Before performance analysis, the implementation was validated through four groups of tests, functional correctness, regression against trusted earlier versions, stability under different block sizes, and OpenMP consistency. For completeness, aggregated outcomes of the correctness, stability, and OpenMP-consistency suites are reported in Appendix A.1, with the corresponding bundle-level summary listed in Appendix A.2.2.

3.2.1 Functional correctness

The first group of tests checked that the public Cholesky routine satisfied the required interface and produced the expected lower-triangular factor on small cases that could be verified directly. The suite included:

- zero-size handling;
- factorisation of a 1×1 matrix;
- the coursework 2×2 example and its log-determinant;
- diagonal symmetric positive-definite matrices;
- known 3×3 and 4×4 reconstruction cases;
- reconstruction of a generated correlation matrix through LL^T ;
- positivity of the diagonal after factorisation;
- rejection of invalid input and early failure on a negative diagonal.

These tests established basic mathematical correctness on both simple analytic examples and representative dense positive-definite matrices.

3.2.2 Regression against trusted and external references

The optimised implementations were checked against internal and external references. Later versions were compared with archived trusted kernels, and the main regression test used correlation matrices with sizes near block boundaries,

$$n \in \{31, 32, 33, 63, 64, 65, 127, 128, 129\},$$

together with several block sizes. For each case:

- the current factor was compared with the archived serial reference;
- the rebuilt matrix LL^T was compared with the original input;
- the maximum absolute difference was recorded.

An additional external-reference check compared the reported log-determinant against `numpy.linalg.slogdet` [7], and the benchmark logs also recorded the maximum absolute and relative errors.

3.2.3 Stability under block-size changes

A separate stability suite checked that changing the block size did not change the numerical result. This had two parts:

- **Repeatability test:** the same factorisation was rerun multiple times for the same matrix and block size, and the resulting factors were compared directly.

- **Block-invariance test:** block sizes

16, 31, 64, 128

were compared against the reference block size 32 using matrix sizes

65, 129, 257.

For each case, the factor was compared with the reference result and LL^T with the original input. Blocking should not change the numerical result [1] [4].

3.2.4 OpenMP consistency

For the parallel versions, an additional test checked that OpenMP parallelisation preserved the same numerical result as the blocked serial formulation. The tested thread counts were

4, 16, 32,

and the tested matrix sizes were

257, 513, 1024.

For each case:

- the implementation was run twice with the same thread count;
- the two parallel factors were compared with each other;
- both were compared with the archived blocked serial reference;
- the rebuilt matrix LL^T was checked against the original matrix.

This verified both numerical consistency and repeatability for the OpenMP implementation.

Performance results

4.1 Validation summary

Before runtime analysis, all canonical milestone bundles passed correctness, stability, and OpenMP consistency checks. An external-reference check against `numpy.linalg.slogdet` also gave `reference_failures=0`, with maximum absolute and relative errors recorded. Factor differences were zero or of order 10^{-14} , indicating floating-point round-off rather than algorithmic deviation, so the runtime differences discussed below are attributed to implementation effects. Expanded validation outcomes are collected in Appendix A.1 and Appendix A.2.2; detailed reference log-determinant checks are reported in Appendix A.2.6.

Version	Correctness	Stability	Consistency	External ref.	Max factor diff.
v0 baseline	Pass	Pass	—	Pass	2.46×10^{-14}
v1 serial Opt	Pass	Pass	—	Pass	2.46×10^{-14}
v2 blocked	Pass	Pass	—	Pass	2.46×10^{-14}
v3 OpenMP	Pass	Pass	Pass	Pass	2.46×10^{-14}
v4 Final Version	Pass	Pass	Pass	Pass	4.71×10^{-14}

Table 4.1: Validation summary for the canonical benchmark versions.

4.2 Performance across versions

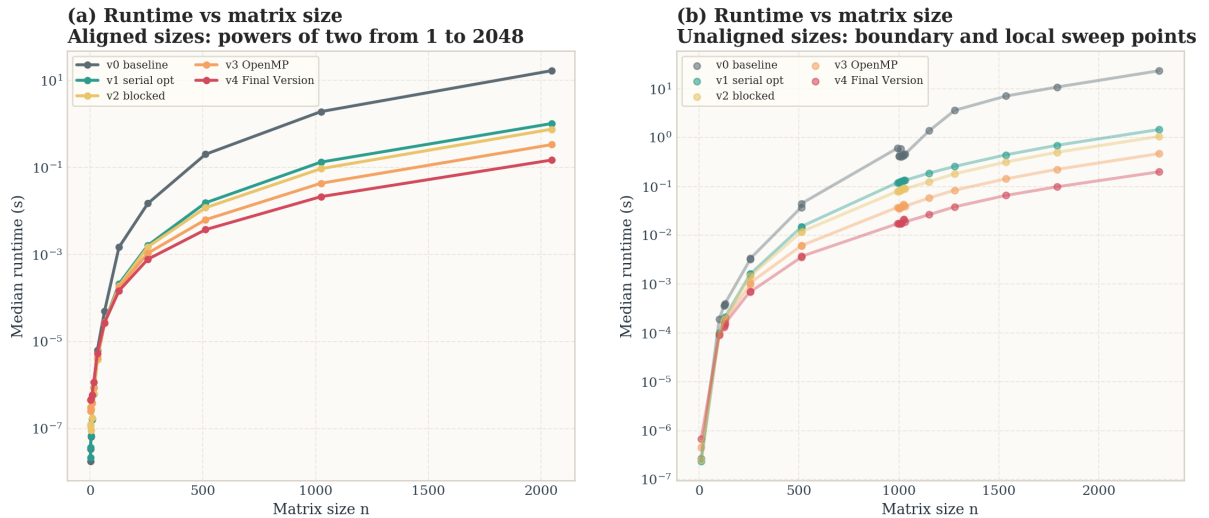


Figure 4.1: Overall runtime across all milestone versions. Panel (a) shows aligned sizes (powers of two from 1 to 2048), panel (b) shows unaligned boundary and local sweep sizes. The runtime axis plotted on a logarithmic scale.

Figure 4.1 shows that all versions behave similarly for small matrices, but separate progressively as matrix size increases, indicating that the benefit of each optimisation stage grows with

problem size. Panel (a) shows aligned matrix sizes and panel (b) shows non-aligned sizes. The logarithmic vertical axis allows the full runtime range to be compared clearly. **Figure 4.2** summarises this effect at representative coarse, fine, and large targets. Panel (a) shows a monotonic runtime reduction from v0 to v4, strongest at the largest size, while panel (b) shows the matching rise in GFLOP/s. Across all versions, runtime remains broadly consistent with the expected $\mathcal{O}(n^3)$ scaling of dense Cholesky factorisation, with the optimisation steps mainly shifting the curves downward rather than changing their asymptotic trend. Overall, the largest gain comes from the first serial clean-up step, with later OpenMP versions providing further improvement. Complete runtime, throughput medians for the coarse, fine, and large sweeps are listed in Appendix A.2.3.

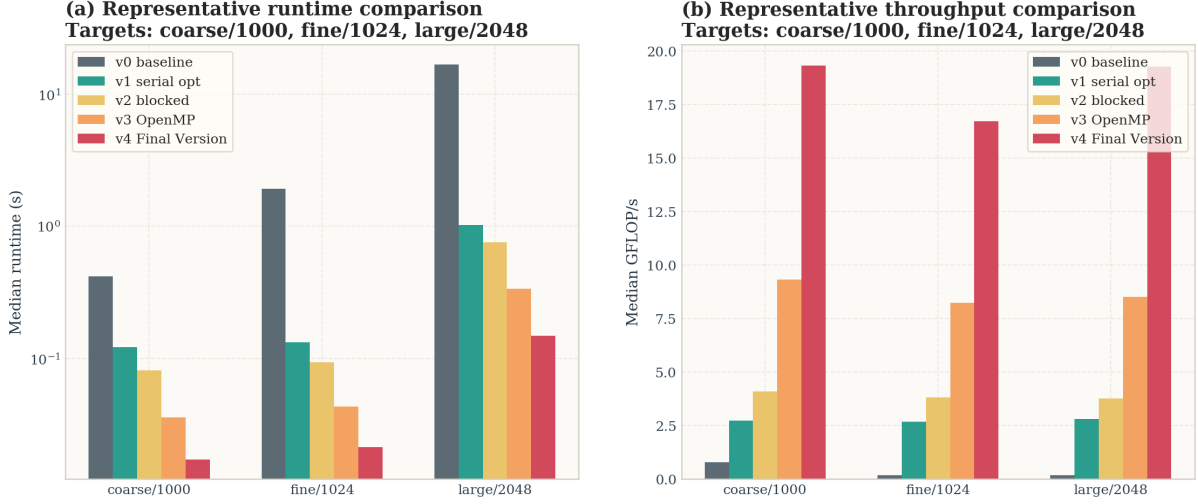


Figure 4.2: representative runtime and throughput across all versions.

4.2.1 v0 to v1: serial kernel clean-up

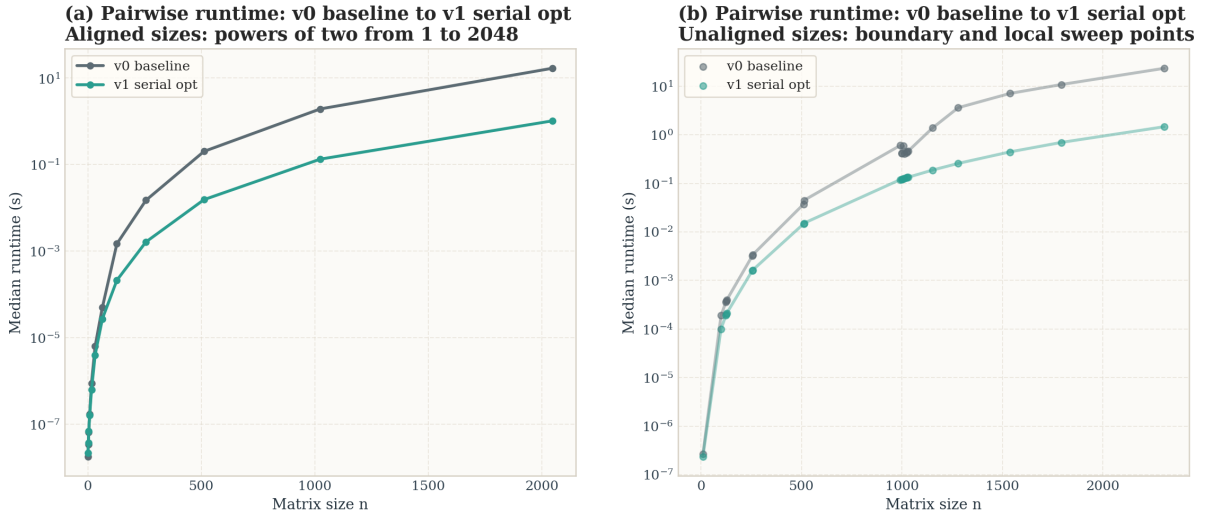


Figure 4.3: Pairwise runtime comparison for v0 and v1.

Figure 4.3 shows the largest single performance change in the optimisation path. Beyond $n = 32$ v1 becomes consistently much faster than v0. At $n = 1024$, the median runtime falls from 1.92s to 0.133s, a speedup of $14.47\times$, and at $n = 2048$ it falls from 16.78s to 1.02s,

or $16.41\times$. The gain is therefore large across full measured range. The unaligned-size data show that v0 is much less stable near boundary sizes, whereas this irregular behaviour largely disappears in v1, consistent with the serial clean-up reducing sensitivity to inefficient memory access.

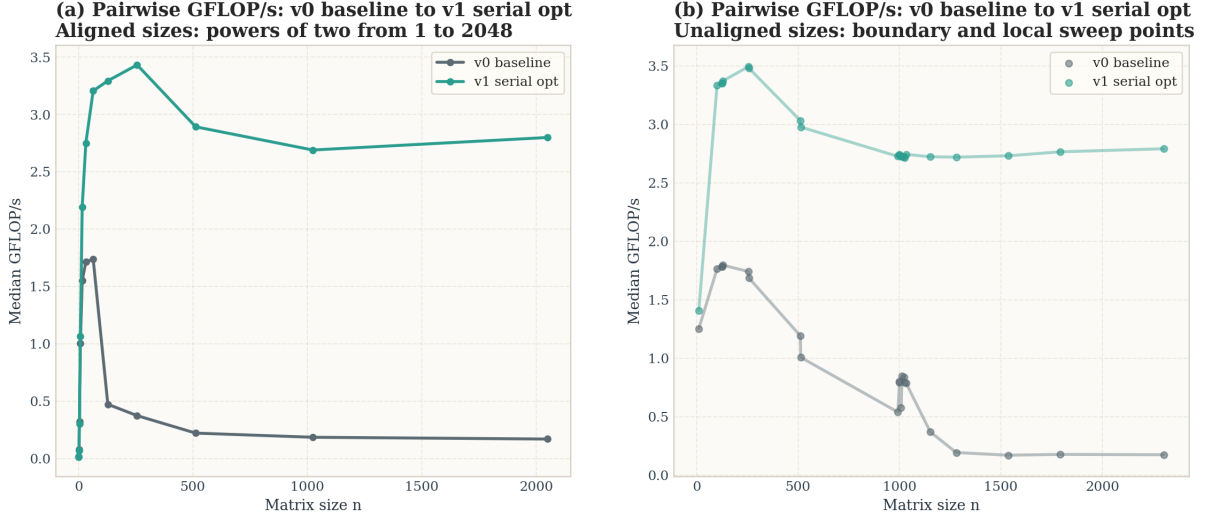


Figure 4.4: throughput comparison between v0 and v1.

Figure 4.4 and **Figure 4.5** support the same hardware-level interpretation. From $n = 1024$, IPC rises from about 0.42 in v0 to about 2.88 in v1, alongside higher throughput. Supporting sweep medians summaries are reported in Appendix A.2.3 and Appendix A.2.7.

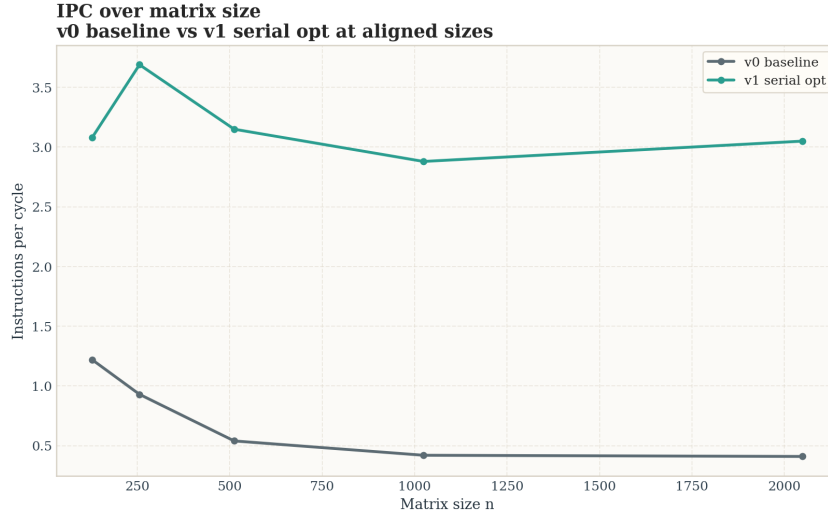


Figure 4.5: IPC with the large jump from v0 to v1.

These changes reduce repeated address arithmetic, replace repeated scalar division with multiplication by `inv_diag`, and make the trailing-update access pattern more regular. Together, they reduce loop overhead and improve inner-kernel efficiency, consistent with the runtime and IPC data.

4.2.2 v1 to v2: blocking for locality

Figure 4.6 shows that the transition from v1 to v2 gives a consistent but smaller improvement than v0 to v1 as shows in two almost overlap. At $n = 1024$, the median runtime falls from 0.133s to 0.093s, giving a speedup of about $1.42\times$. At $n = 2048$, the runtime improves from 1.02s to 0.758s.

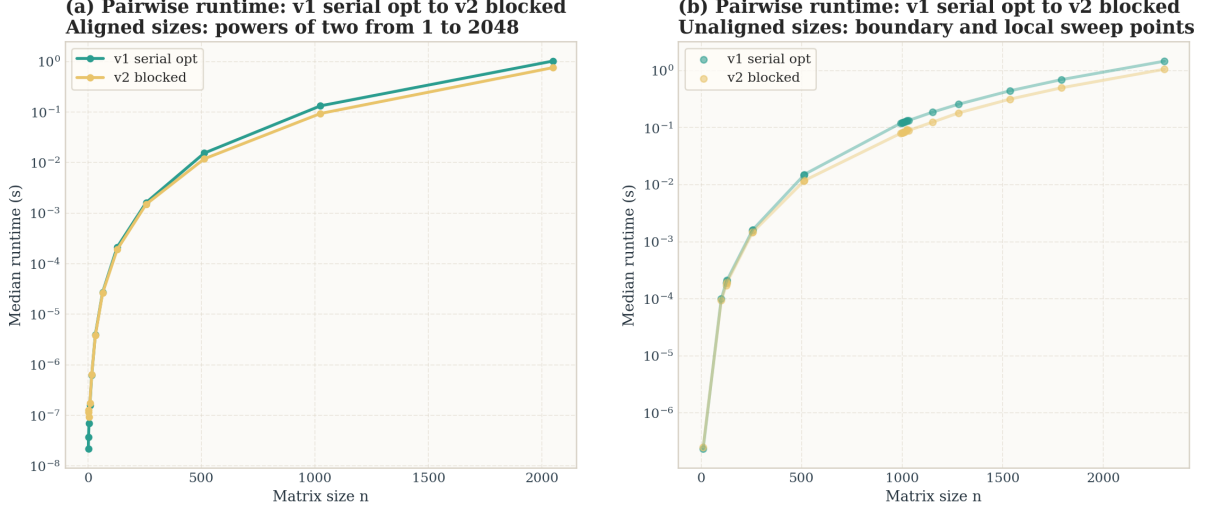


Figure 4.6: Pairwise runtime comparison for v1 and v2.

Version v2 reorganises the kernel into a blocked formulation, but does not change the underlying $\mathcal{O}(n^3)$ arithmetic. The measured gain is then moderated, this is consistent with blocking improving cache locality.

4.2.3 v2 to v3: OpenMP on the trailing update

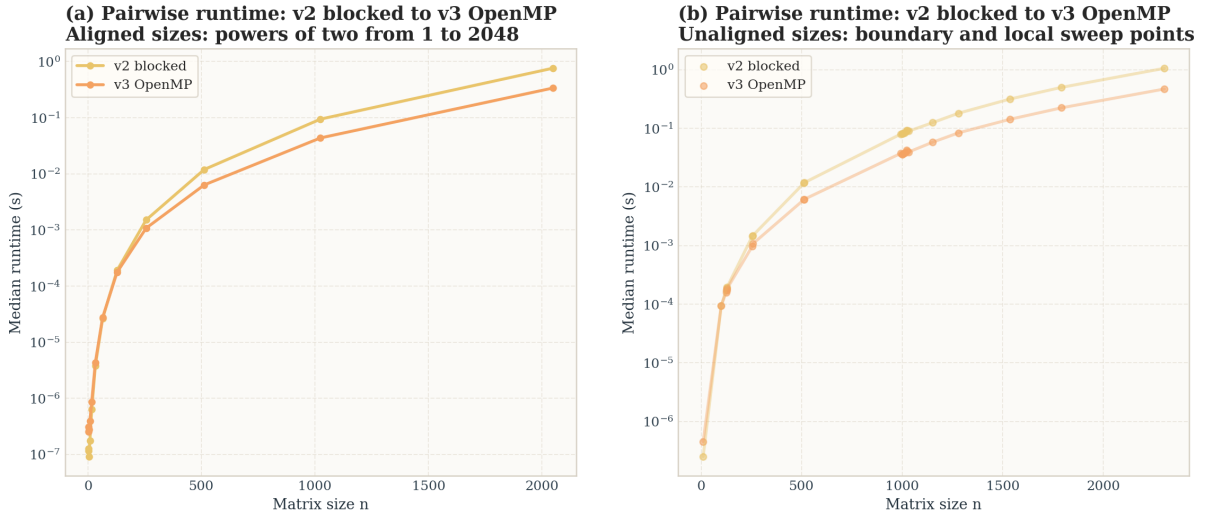


Figure 4.7: Pairwise runtime comparison for v2 and v3.

Figure 4.7 shows the first major gain from OpenMP parallelisation. At $n = 1024$, the runtime improves from 0.0938s in v2 to 0.0434s in v3, which is about $2.16\times$. At the larger case $n = 2048$, the improvement shows larger, from 0.758s to 0.336s, or about $2.26\times$. This is the cleanest first

test of OpenMP because the main change from v2 to v3 is only thread-level parallelism in the trailing update stage.

This pattern is consistent with the v3 implementation change. The blocked trailing update was parallelised with OpenMP, because tiles are independent for fixed panel. As matrix size increases, this stage accounts for a larger share of the total $\mathcal{O}(n^3)$ work, so the stronger gains at larger sizes consistent with a more effective parallel kernel.

4.2.4 v3 to v4: refined parallel kernel

Figure 4.8 shows that the transition from v3 to v4 gives another substantial improvement. At $n = 1024$, runtime falls from 0.0434s in v3 to 0.0214s in the v4 Final Version, corresponding to about $2.03\times$ speedup. At $n = 2048$, runtime falls from 0.336s to 0.149s, or about $2.26\times$. The gain is consistent with a more effective parallel kernel.

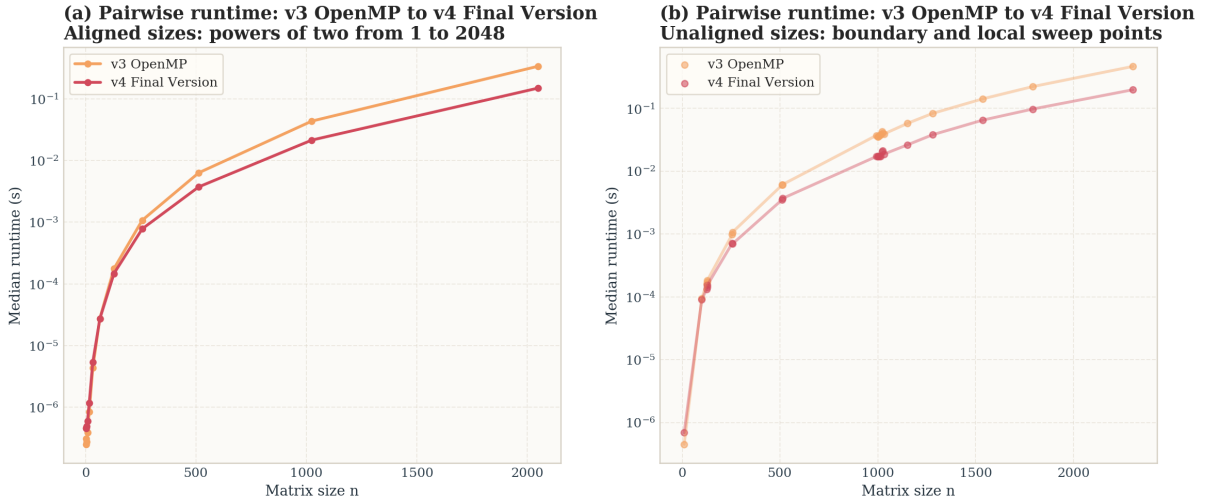


Figure 4.8: Pairwise runtime comparison for v3 and the v4 Final Version.

The v4 Final Version keeps the blocked OpenMP structure but further improves through runtime-configurable scheduling, parallel block-column solves for sufficiently large remaining problems, and `omp simd` reductions in the block-column and trailing-update dot-product kernels. These changes target the main overheads left by v3, load imbalance in the irregular triangular update, serial cost between panel and trailing work, limited data-level parallelism.

4.3 Effect of OpenMP

4.3.1 Scaling with thread count at a fixed problem size

At the canonical fixed size $n = 1024$, both OpenMP versions show real thread scaling, but v4 extracts a larger benefit from the same thread budget. This subsection is used as the fixed-size reference case for the report. It shows that v4 is the stronger implementation under the standard workload, but the broader question of size dependence is addressed in the next subsection.

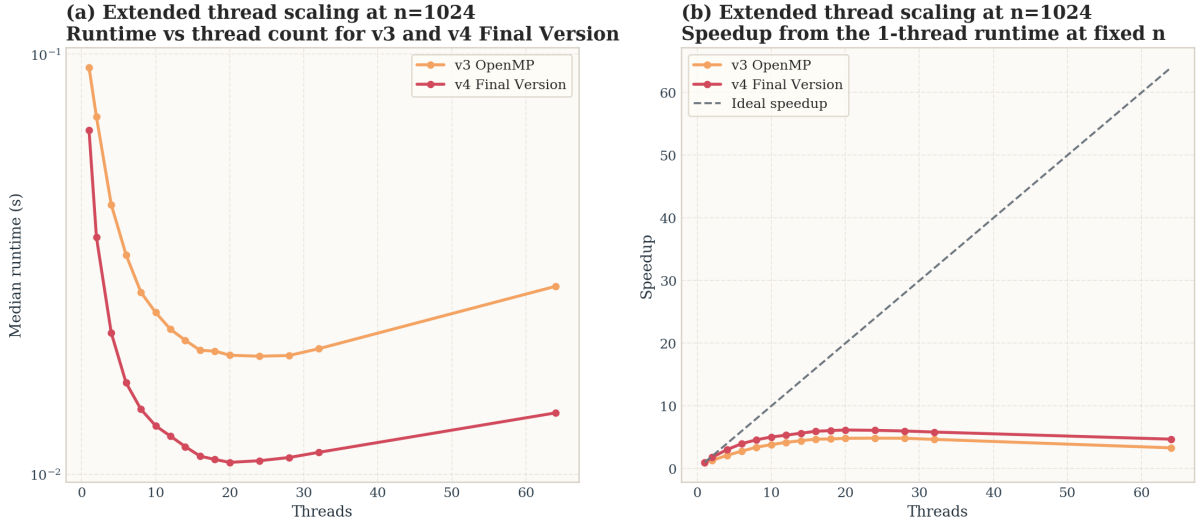


Figure 4.9: Extended thread scaling at $n = 1024$. Panel (a) shows runtime and panel (b) shows speedup relative to 1 thread.

Figure 4.9 shows that runtime falls over the useful scaling range for both versions, but more strongly for v4. For v3, runtime drops from 0.0929s at 1 thread to 0.0197s at 16 threads, with a best value of 0.0191s at 24 threads. For v4, runtime drops from 0.0658s to 0.0110s at 16 threads and reaches its best value of 0.0107s at 20 threads, after which the curve flattens and weakens slightly by 64 threads. Because the extended sweep remains within physical cores on the icelake platform (see §3.1.1), this high-thread flattening beyond 32 threads is not caused by a transition into SMT. The speedup curves show the same ordering: at 16 threads, v3 reaches $4.71\times$ while v4 reaches $5.96\times$, and the peak speedup rises from $4.86\times$ to $6.17\times$. The canonical $n = 1024$ case therefore shows a finite useful scaling range, with v4 sustaining better thread-level performance under the standard problem size.

Version	1-thread runtime (s)	16-thread runtime (s)	Peak speedup	Peak thread	Karp–Flatt at 16 threads
v3 OpenMP	0.0929	0.0197	$4.86\times$	24	0.160
v4 Final	0.0658	0.0110	$6.17\times$	20	0.112

Table 4.2: Fixed-size thread-scaling summary at the canonical $n = 1024$ case.

Table 4.2 summarises the same result numerically. Peak speedup captures the best observed benefit for each version, while the 16-thread point is retained as a fixed comparison point before stronger high-thread saturation with all efficiency and Karp–Flatt values are computed from the median runtime at the same fixed problem size and thread count. On this basis, v4 also shows a lower apparent non-parallel component, with the Karp–Flatt serial fraction decreasing

from 0.160 to 0.112. The underlying medians at the canonical $n = 1024$ case are listed in Appendix A.2.4, the scaling metrics reported in Appendix A.2.8, Tables 20 and 21. The next subsection tests this thread benefit increases as the available blocked work grows.

4.3.2 Dependence of thread benefit on matrix size

Thread benefit depends strongly on matrix size. Small matrix size are overhead-dominated, whereas larger size gain more useful parallel work and delay the saturation.

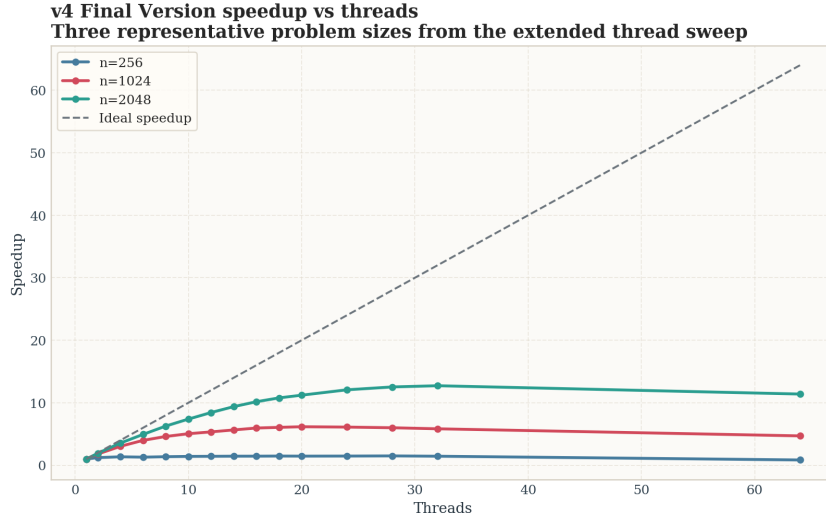


Figure 4.10: v4 speedup versus thread count for $n = 256$, 1024, and 2048. Larger matrices achieve higher speedup and retain useful scaling to higher thread counts.

Figure 4.10 shows the size dependence directly. As n increases, the speedup curve shifts upward and the onset of saturation moves to higher thread counts. At $n = 256$, speedup remains close to unity and falls below $1\times$ at 64 threads, indicating that fixed parallel overhead dominates the useful work. At $n = 1024$, the same implementation sustains substantially stronger scaling and reaches $5.96\times$ speedup at 16 threads. At $n = 2048$, the gain is stronger again, reaching $10.17\times$ at 16 threads and $12.73\times$ at 32 threads.

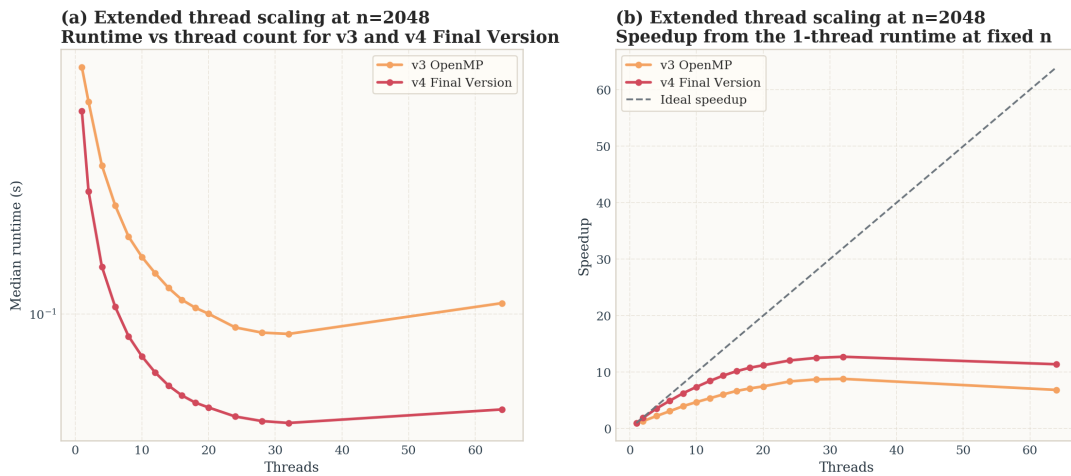


Figure 4.11: Extended thread scaling at $n = 2048$. Panel (a) shows runtime and panel (b) shows speedup relative to 1 thread.

Figure 4.11 and **Table 4.3** together show the strongest size-dependent scaling in the study. At $n = 2048$, the representative large-problem case, v3 continues to improve up to 32 threads and reaches $8.81\times$ speedup, while v4 rises to $10.17\times$ at 16 threads and $12.73\times$ at 32 threads. Table 4.3 shows that part of a clear cross-size pattern rather than isolated case. Peak speedup is reported to capture the best attainable benefit at each size, while the 16-thread results are retained as a common reference point before the strongest high-thread saturation. On this fixed basis, v4 speedup rises from $1.46\times$ at $n = 256$ to $5.96\times$ at $n = 1024$ and $10.17\times$ at $n = 2048$, efficiency increases from 0.09 to 0.37 and 0.64, and the Karp–Flatt serial fraction falls. The combined evidence shows that larger matrices retain more useful parallel work and allow v4 to extract substantially greater thread benefit.

n	v3 peak speedup	v4 peak speedup	v4 peak thread	v4 speedup at 16 threads	v4 efficiency at 16 threads	v4 Karp–Flatt at 16 threads
256	$1.39\times$	$1.50\times$	28	$1.46\times$	0.09	0.665
1024	$4.86\times$	$6.17\times$	20	$5.96\times$	0.37	0.112
2048	$8.81\times$	$12.73\times$	32	$10.17\times$	0.64	0.038

Table 4.3: Cross-size scaling summary. Peak speedup and peak thread report the best observed benefit at each size.

Overall, the figure and table show as n increases, the blocked trailing update occupies a larger share of total work, fixed OpenMP overhead is amortised more effectively, and saturation is pushed to higher thread counts. The $n = 256$ case is therefore best interpreted as a negative control. The full cross-size thread-scaling data are reported in Appendix A.2.8 (Tables 20 and 21), which list the derived runtime, throughput, speedup, efficiency, and Karp–Flatt metrics for $n = 256$, 1024, and 2048.

4.3.3 Schedule sensitivity

Blocked trailing update has an irregular triangular workload, so tiles contain different amounts of work and load balance depends on how iterations are assigned across threads. **Figure 4.12** shows that this effect is small in v3 but much stronger in v4. The v3 runtime changes little across policies, v4 varies clearly.

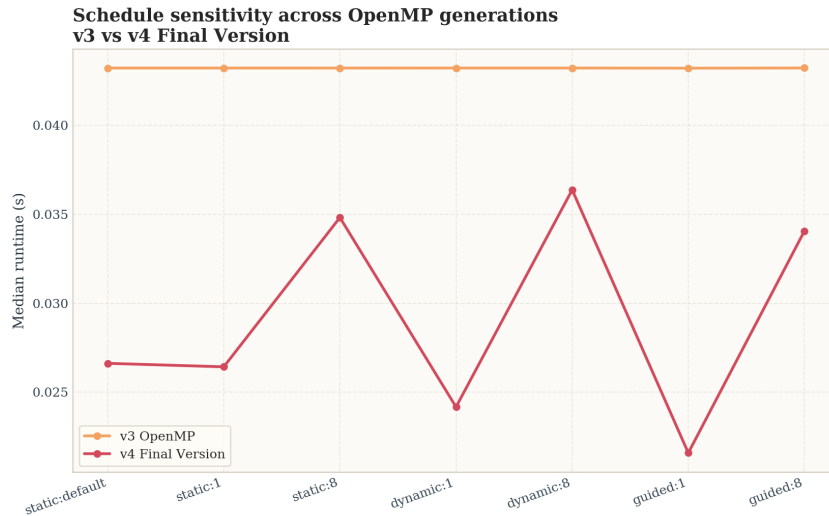


Figure 4.12: Schedule sensitivity across OpenMP generations.

Figure 4.13 makes the v4 behaviour clearer. The best result is obtained with `guided,1`, with `dynamic,1` close behind, while larger chunk sizes are slower. This is consistent with finer-grained scheduling improving load balance in the uneven triangular tile work. Complete schedule-study medians are reported in Appendix A.2.5.

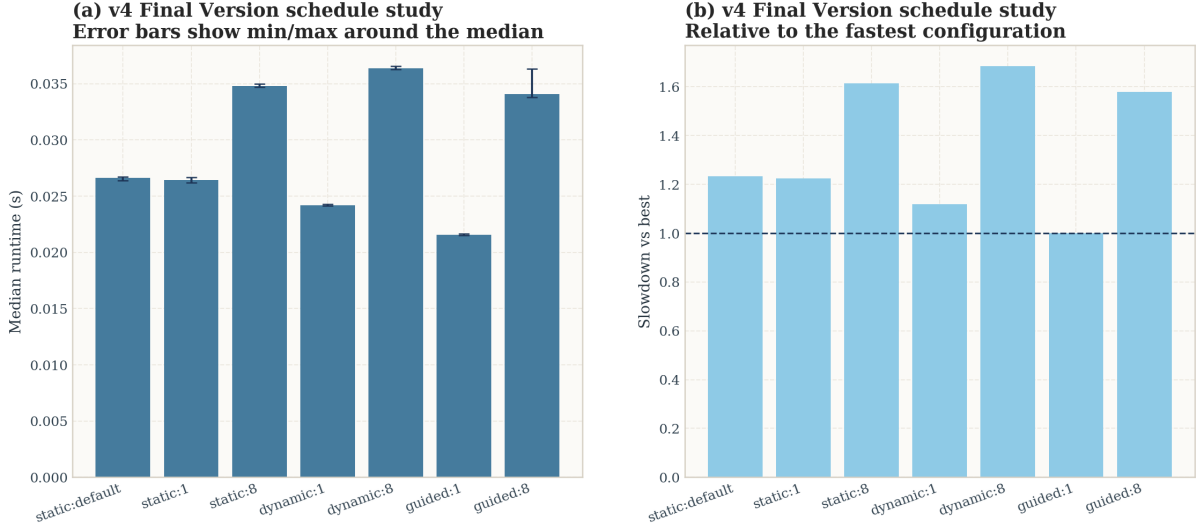


Figure 4.13: Schedule study for the v4 Final Version at $n = 1024$ with 4 threads. The best result is obtained with `guided,1`.

4.3.4 Summary

Overall, the OpenMP results show that thread-level speedup is real but strongly size-dependent. At the canonical $n = 1024$ case, both versions scale over a finite useful thread range, with v4 outperforming v3. The cross-size results show that this benefit when small matrices are dominated by fixed parallel overhead, but larger matrices retain more useful blocked work and scale to higher thread counts. The schedule study provides secondary support at the kernel-tuning level.

Discussion and conclusion

The v0 to v1 step preserved the algorithm but reduced overhead and made memory access more regular, showing that the baseline was limited mainly by implementation efficiency.

The move from v1 to v2 gave a smaller but consistent gain, indicating improved locality from blocking. For larger matrices, OpenMP delivered the main parallel benefit through the blocked trailing update, and the further gain from v3 to v4 reflects a stronger parallel kernel.

The OpenMP results also showed that thread benefit was strongly size-dependent. Small matrices were overhead-dominated, whereas larger matrices retained more useful blocked work and scaled further, although scaling remained sublinear and eventually saturated at high thread counts.

Overall, the project produced a reproducible Cholesky factorization for single-node CSD3 execution.

Appendix

.1 Full validation results

Version	Correctness suite	Regression against archived reference	Max factor diff.
v0	11/11 passed. Same functional test set used for all versions.	27/27 passed. Tested at block sizes {32, 64, 128} and sizes {31, 32, 33, 63, 64, 65, 127, 128, 129}.	1.82×10^{-14}
v1	11/11 passed. Same functional test set used for all versions.	27/27 passed. Tested at block sizes {32, 64, 128} and sizes {31, 32, 33, 63, 64, 65, 127, 128, 129}.	1.82×10^{-14}
v2	11/11 passed. Same functional test set used for all versions.	27/27 passed. Tested at block sizes {32, 64, 128} and sizes {31, 32, 33, 63, 64, 65, 127, 128, 129}.	1.82×10^{-14}
v3	11/11 passed. Same functional test set used for all versions.	27/27 passed. Tested at block sizes {32, 64, 128} and sizes {31, 32, 33, 63, 64, 65, 127, 128, 129}.	1.82×10^{-14}
v4	11/11 passed. Same functional test set used for all versions.	27/27 passed. Tested at block sizes {32, 64, 128} and sizes {31, 32, 33, 63, 64, 65, 127, 128, 129}.	1.82×10^{-14}

Table 1: Correctness and regression summary for all canonical versions.

Version	Stability suite	OpenMP consistency suite	Max factor diff.
v0	27/27 passed. Repeatability tested for block sizes {16, 31, 32, 64, 128} at $n \in \{65, 129, 257\}$; block invariance tested against reference block size 32.	9/9 passed. Tested at thread counts {1, 2, 4} and sizes {65, 129, 257}.	4.71×10^{-14}
v1	27/27 passed. Repeatability tested for block sizes {16, 31, 32, 64, 128} at $n \in \{65, 129, 257\}$; block invariance tested against reference block size 32.	9/9 passed. Tested at thread counts {1, 2, 4} and sizes {65, 129, 257}.	4.71×10^{-14}
v2	27/27 passed. Repeatability tested for block sizes {16, 31, 32, 64, 128} at $n \in \{65, 129, 257\}$; block invariance tested against reference block size 32.	9/9 passed. Tested at thread counts {1, 2, 4} and sizes {65, 129, 257}.	4.71×10^{-14}
v3	27/27 passed. Repeatability tested for block sizes {16, 31, 32, 64, 128} at $n \in \{65, 129, 257\}$; block invariance tested against reference block size 32.	9/9 passed. Tested at thread counts {4, 16, 32} and sizes {257, 513, 1024}.	4.71×10^{-14}
v4	27/27 passed. Repeatability tested for block sizes {16, 31, 32, 64, 128} at $n \in \{65, 129, 257\}$; block invariance tested against reference block size 32.	9/9 passed. Tested at thread counts {4, 16, 32} and sizes {257, 513, 1024}.	4.71×10^{-14}

Table 2: Stability and OpenMP consistency summary for all canonical versions.

.2 Appendix: Canonical benchmark result tables

This appendix summarises the canonical benchmark bundles stored under `history/results/` for the five versions.

.2.1 Benchmark bundle overview

Version	Hostname	Compiler	Threads	Block	Schedule	Chunk	Partition
v0	cpu-q-66	GNU 8.5.0	1	32	static	0	icelake
v1	cpu-q-66	GNU 8.5.0	1	32	static	0	icelake
v2	cpu-q-66	GNU 8.5.0	1	32	static	0	icelake
v3	cpu-q-66	GNU 8.5.0	4	32	static	0	icelake
v4	cpu-q-66	GNU 8.5.0	4	32	guided	1	icelake

Table 3: Canonical benchmark bundles and their recorded runtime settings.

.2.2 Validation summary

Version	Correctness	Against v1 max diff.	Stability max diff.	OMP consistency max diff.
v0	11/11 pass	1.82×10^{-14}	1.82×10^{-14}	4.71×10^{-14}
v1	11/11 pass	1.82×10^{-14}	1.82×10^{-14}	4.71×10^{-14}
v2	11/11 pass	1.82×10^{-14}	1.82×10^{-14}	4.71×10^{-14}
v3	11/11 pass	1.82×10^{-14}	1.82×10^{-14}	4.71×10^{-14}
v4	11/11 pass	1.82×10^{-14}	1.82×10^{-14}	4.71×10^{-14}

Table 4: Validation summary from the canonical result bundles.

.2.3 Full sweep tables

n	v0	v1	v2	v3	v4
1	0.000000	0.000000	0.000000	0.000000	0.000000
2	0.000000	0.000000	0.000000	0.000000	0.000000
4	0.000000	0.000000	0.000000	0.000000	0.000000
8	0.000000	0.000000	0.000000	0.000000	0.000001
10	0.000000	0.000000	0.000000	0.000000	0.000001
16	0.000001	0.000001	0.000001	0.000001	0.000001
32	0.000006	0.000004	0.000004	0.000004	0.000005
64	0.000050	0.000027	0.000026	0.000028	0.000027
100	0.000189	0.000100	0.000093	0.000095	0.000090
124	0.000356	0.000190	0.000173	0.000159	0.000131
127	0.000382	0.000203	0.000186	0.000173	0.000142
128	0.001476	0.000212	0.000191	0.000177	0.000147
129	0.000398	0.000212	0.000195	0.000184	0.000155
255	0.003173	0.001581	0.001454	0.000979	0.000710
256	0.014927	0.001629	0.001507	0.001076	0.000781
257	0.003353	0.001625	0.001479	0.001070	0.000697
511	0.037323	0.014664	0.011573	0.006057	0.003540
512	0.201277	0.015466	0.011907	0.006325	0.003738
513	0.044652	0.015112	0.011796	0.006147	0.003695
1000	0.419113	0.121694	0.081512	0.035693	0.017237
1024	1.994010	0.133054	0.094294	0.043483	0.021442

Table 5: Coarse sweep medians for runtime (s).

n	v0	v1	v2	v3	v4
1	0.0185	0.0152	0.0029	0.0011	0.0007
2	0.0784	0.0721	0.0212	0.0105	0.0057
4	0.3232	0.3048	0.2319	0.0773	0.0433
8	1.0039	1.0667	0.9697	0.4354	0.2854
10	1.2531	1.4065	1.3228	0.7407	0.4859
16	1.5515	2.1951	2.1367	1.5950	1.1620
32	1.7144	2.7513	2.8699	2.5075	2.0261
64	1.7398	3.2062	3.3080	3.1737	3.2088
100	1.7638	3.3358	3.5715	3.5145	3.6871
124	1.7849	3.3517	3.6731	3.9896	4.8483
127	1.7863	3.3553	3.6655	3.9433	4.8028
128	0.4735	3.2935	3.6670	3.9486	4.7453
129	1.7966	3.3723	3.6624	3.8889	4.6283
255	1.7418	3.4952	3.8015	5.6446	7.7810
256	0.3747	3.4324	3.7101	5.1982	7.1595
257	1.6877	3.4817	3.8253	5.2873	8.1236
511	1.1917	3.0332	3.8433	7.3426	12.5632
512	0.2223	2.8927	3.7574	7.0730	11.9695
513	1.0078	2.9780	3.8151	7.3213	12.1787
1000	0.7953	2.7391	4.0894	9.3388	19.3386
1024	0.1795	2.6900	3.7957	8.2312	16.6922

Table 6: Coarse sweep medians for GFLOP/s.

n	v0	v1	v2	v3	v4
992	0.603118	0.119318	0.079341	0.037176	0.017168
1000	0.416383	0.121590	0.081264	0.035614	0.017243
1008	0.589807	0.125140	0.083252	0.037153	0.017319
1013	0.409517	0.127238	0.084875	0.038335	0.017328
1023	0.426065	0.130873	0.091059	0.042727	0.020629
1024	1.923412	0.133038	0.093859	0.043428	0.021400
1025	0.452215	0.132158	0.092116	0.040843	0.021418
1032	0.463845	0.133498	0.089171	0.039373	0.018819

Table 7: Fine sweep medians for runtime (s).

n	v0	v1	v2	v3	v4
992	0.5395	2.7271	4.1012	8.7530	18.9540
1000	0.8005	2.7415	4.1019	9.3596	19.3314
1008	0.5788	2.7281	4.1008	9.1890	19.7128
1013	0.8461	2.7233	4.0825	9.0387	19.9968
1023	0.8376	2.7268	3.9191	8.3522	17.2995
1024	0.1861	2.6903	3.8133	8.2416	16.7251
1025	0.7938	2.7162	3.8969	8.7888	16.7596
1032	0.7899	2.7444	4.1086	9.3052	19.4682

Table 8: Fine sweep medians for GFLOP/s.

n	v0	v1	v2	v3	v4
1152	1.378322	0.187081	0.125081	0.057994	0.026397
1280	3.607288	0.256956	0.180767	0.083568	0.038191
1536	7.065063	0.442150	0.315443	0.142347	0.065226
1792	10.787767	0.693467	0.498703	0.223255	0.098025
2048	16.784578	1.022846	0.757848	0.335719	0.148537
2304	23.322760	1.460010	1.054230	0.467812	0.198819

Table 9: Large sweep medians for runtime (s).

n	v0	v1	v2	v3	v4
1152	0.3697	2.7240	4.0742	8.7872	19.3053
1280	0.1938	2.7205	3.8671	8.3650	18.3039
1536	0.1710	2.7320	3.8294	8.4860	18.5195
1792	0.1778	2.7661	3.8464	8.5919	19.5684
2048	0.1706	2.7994	3.7782	8.5289	19.2768
2304	0.1748	2.7924	3.8671	8.7147	20.5054

Table 10: Large sweep medians for GFLOP/s.

.2.4 OpenMP thread scaling

Threads	v0	v1	v2	v3	v4
1	2.113434	0.134770	0.094073	0.093005	0.066826
2	2.113437	0.134802	0.094048	0.070740	0.036949
4	2.113359	0.134783	0.094054	0.043241	0.021731
16	2.113402	0.134773	0.094072	0.020539	0.011059
32	2.113370	0.134762	0.094057	0.019861	0.010697

Table 11: Thread-scaling medians at the canonical $n = 1024$ benchmark size: runtime.

Threads	v0	v1	v2	v3	v4
1	0.1694	2.6557	3.8046	3.8484	5.3559
2	0.1694	2.6551	3.8056	5.0596	9.6866
4	0.1694	2.6555	3.8054	8.2772	16.4700
16	0.1694	2.6557	3.8047	17.4258	32.3628
32	0.1694	2.6559	3.8053	18.0207	33.4594

Table 12: Thread-scaling medians at the canonical $n = 1024$ benchmark size: throughput.

.2.5 Schedule study at $n = 1024$, 4 threads

Schedule, chunk	v0	v1	v2	v3	v4
static, default	2.113498	0.134866	0.094003	0.043225	0.026624
static, 1	2.113384	0.134821	0.094000	0.043225	0.026427
static, 8	2.113060	0.134812	0.094006	0.043225	0.034820
dynamic, 1	2.113054	0.134835	0.094020	0.043226	0.024172
dynamic, 8	2.113046	0.134817	0.093995	0.043226	0.036376
guided, 1	2.113080	0.134825	0.093992	0.043219	0.021587
guided, 8	2.113009	0.134818	0.093999	0.043227	0.034064

Table 13: Schedule-study medians from the canonical result bundles: runtime.

Schedule, chunk	v0	v1	v2	v3	v4
static, default	0.1693	2.6539	3.8075	8.2803	13.4435
static, 1	0.1694	2.6547	3.8076	8.2802	13.5433
static, 8	0.1694	2.6549	3.8073	8.2803	10.2790
dynamic, 1	0.1694	2.6545	3.8068	8.2800	14.8069
dynamic, 8	0.1694	2.6548	3.8078	8.2800	9.8392
guided, 1	0.1694	2.6547	3.8079	8.2815	16.5798
guided, 8	0.1694	2.6548	3.8076	8.2798	10.5071

Table 14: Schedule-study medians from the canonical result bundles: throughput.

.2.6 Reference log-determinant checks

n	v0	v1	v2	v3	v4
2	1.04×10^{-17}	1.04×10^{-17}	1.04×10^{-17}	1.04×10^{-17}	1.04×10^{-17}
8	5.33×10^{-15}	8.88×10^{-15}	8.88×10^{-15}	8.88×10^{-15}	8.88×10^{-15}
32	2.84×10^{-14}	7.11×10^{-14}	7.11×10^{-14}	7.11×10^{-14}	7.11×10^{-14}
128	1.14×10^{-13}	0	3.41×10^{-13}	3.41×10^{-13}	2.27×10^{-13}
512	1.36×10^{-12}	4.55×10^{-13}	4.55×10^{-13}	4.55×10^{-13}	9.09×10^{-13}
1024	9.09×10^{-13}	3.64×10^{-12}	1.82×10^{-12}	1.82×10^{-12}	2.73×10^{-12}

Table 15: Reference log-determinant absolute errors from the canonical result bundles.

n	v0	v1	v2	v3	v4
2	0.000000	0.000000	0.000001	0.000003	0.000004
8	0.000001	0.000001	0.000001	0.000004	0.000003
32	0.000008	0.000005	0.000006	0.000009	0.000010
128	0.001444	0.000193	0.000198	0.000197	0.000160
512	0.191438	0.014293	0.012004	0.011894	0.008546
1024	1.894067	0.121423	0.093576	0.092432	0.064503

Table 16: Reference log-determinant check elapsed time (s) from the canonical result bundles.

.2.7 Representative hardware-counter summaries

n	v0	v1	v2	v3	v4
128	1.220	3.080	2.530	2.360	2.520
256	0.930	3.690	2.680	2.620	2.950
512	0.540	3.150	2.650	2.660	3.130
1000	1.960	3.000	2.880	2.910	3.900
1024	0.420	2.880	2.680	2.700	3.290
1536	0.420	2.960	2.690	2.720	3.330
1792	0.440	3.030	2.710	2.740	3.400
2048	0.410	3.050	2.640	2.670	3.280
2304	0.430	3.040	2.710	2.740	3.430

Table 17: Representative IPC values from `perf_basic.csv`.

n	v0	v1	v2	v3	v4
128	47.363	46.701	47.478	44.966	44.396
256	17.093	16.294	12.060	20.144	34.338
512	0.016	0.151	6.059	3.470	2.651
1000	0.087	0.123	3.352	3.294	3.466
1024	0.002	0.022	0.170	0.201	0.302
1536	0.015	0.223	1.640	1.604	1.618
1792	0.015	0.193	1.255	1.240	1.266
2048	0.031	0.270	1.224	1.264	1.298
2304	0.095	0.395	2.292	2.170	2.169

Table 18: Representative cache-miss % values from `perf_basic.csv`.

n	v0	v1	v2	v3	v4
128	4.60×10^7	9.72×10^6	8.78×10^6	9.48×10^6	8.72×10^6
256	4.46×10^8	5.37×10^7	4.97×10^7	5.09×10^7	4.17×10^7
512	6.05×10^9	4.75×10^8	3.69×10^8	3.69×10^8	2.77×10^8
1000	1.23×10^{10}	3.64×10^9	2.45×10^9	2.43×10^9	1.58×10^9
1024	6.20×10^{10}	4.06×10^9	2.82×10^9	2.81×10^9	2.00×10^9
1536	2.09×10^{11}	1.33×10^{10}	9.40×10^9	9.32×10^9	6.58×10^9
1792	3.16×10^{11}	2.05×10^{10}	1.48×10^{10}	1.47×10^{10}	1.02×10^{10}
2048	5.05×10^{11}	3.04×10^{10}	2.26×10^{10}	2.24×10^{10}	1.57×10^{10}
2304	6.88×10^{11}	4.34×10^{10}	3.14×10^{10}	3.11×10^{10}	2.13×10^{10}

Table 19: Representative cycles values from `perf_basic.csv`.

.2.8 Extended thread-scaling derived metrics

n	Threads	Runtime (s)	GFLOP/s	Speedup	Efficiency	Karp–Flatt ε
256	1	0.001503	3.7216	1.000000	1.000000	–
	2	0.001418	3.9451	1.060075	0.530037	0.886659
	4	0.001168	4.7864	1.286126	0.321531	0.703372
	6	0.001141	4.9019	1.317155	0.219526	0.711054
	8	0.001085	5.1543	1.384980	0.173122	0.682323
	10	0.001082	5.1677	1.388585	0.138858	0.689064
	12	0.001091	5.1252	1.377179	0.114765	0.701224
	14	0.001103	5.0695	1.362205	0.097300	0.713650
	16	0.001085	5.1565	1.385586	0.086599	0.703164
	18	0.001103	5.0691	1.362101	0.075672	0.718522
	20	0.001106	5.0566	1.358740	0.067937	0.722080
	24	0.001104	5.0640	1.360711	0.056696	0.723384
	28	0.001111	5.0319	1.352092	0.048289	0.729950
	32	0.001151	4.8576	1.305267	0.040790	0.758582
	64	0.001695	3.2997	0.886641	0.013854	1.129881
1024	1	0.092934	3.8513	1.000000	1.000000	–
	2	0.070919	5.0468	1.310416	0.655208	0.526233
	4	0.043862	8.1599	2.118756	0.529689	0.295967
	6	0.033293	10.7505	2.791411	0.465235	0.229890
	8	0.027132	13.1916	3.425236	0.428154	0.190801
	10	0.024268	14.7486	3.829533	0.382953	0.179032
	12	0.022137	16.1678	4.198022	0.349835	0.168954
	14	0.020846	17.1697	4.458160	0.318440	0.164639
	16	0.019733	18.1383	4.709669	0.294354	0.159818
	18	0.019645	18.2193	4.730710	0.262817	0.164996
	20	0.019208	18.6334	4.838235	0.241912	0.164934
	24	0.019104	18.7352	4.864652	0.202694	0.171024
	28	0.019176	18.6646	4.846331	0.173083	0.176947
	32	0.019900	17.9854	4.669969	0.145937	0.188784
	64	0.028014	12.7761	3.317370	0.051834	0.290355
2048	1	0.748347	3.8262	1.000000	1.000000	–
	2	0.564098	5.0759	1.326628	0.663314	0.507582
	4	0.335666	8.5302	2.229439	0.557360	0.264724
	6	0.242067	11.8286	3.091489	0.515248	0.188162
	8	0.187870	15.2409	3.983332	0.497917	0.144053

n	Threads	Runtime (s)	GFLOP/s	Speedup	Efficiency	Karp–Flatt ε
	10	0.159204	17.9852	4.700563	0.470056	0.125267
	12	0.139297	20.5555	5.372332	0.447694	0.112152
	14	0.123574	23.1709	6.055875	0.432563	0.100908
	16	0.112215	25.5164	6.668898	0.416806	0.093280
	18	0.105195	27.2191	7.113909	0.395217	0.090015
	20	0.100089	28.6075	7.476788	0.373839	0.088155
	24	0.089552	31.9738	8.356577	0.348191	0.081391
	28	0.085842	33.3555	8.717699	0.311346	0.081921
	32	0.084939	33.7102	8.810399	0.275325	0.084906
	64	0.109211	26.2182	6.852319	0.107067	0.132379

Table 20: Extended thread-scaling metrics for v3 OpenMP at $n \in \{256, 1024, 2048\}$. Speedup is computed against the 1-thread runtime at fixed n , efficiency is $S(p)/p$, and the Karp–Flatt serial fraction is $\varepsilon(p) = \frac{1/S(p)-1/p}{1-1/p}$ for $p > 1$.

n	Threads	Runtime (s)	GFLOP/s	Speedup	Efficiency	Karp–Flatt ε
256	1	0.001103	5.0700	1.000000	1.000000	–
	2	0.000903	6.1965	1.222198	0.611099	0.636396
	4	0.000812	6.8851	1.358026	0.339506	0.648484
	6	0.000850	6.5817	1.298168	0.216361	0.724379
	8	0.000804	6.9560	1.372012	0.171502	0.690122
	10	0.000778	7.1915	1.418455	0.141845	0.672214
	12	0.000769	7.2750	1.434918	0.119576	0.669350
	14	0.000758	7.3793	1.455501	0.103964	0.662976
	16	0.000757	7.3903	1.457655	0.091103	0.665102
	18	0.000747	7.4853	1.476401	0.082022	0.658342
	20	0.000753	7.4223	1.463976	0.073199	0.666391
	24	0.000748	7.4781	1.474977	0.061457	0.663976
	28	0.000736	7.6013	1.499274	0.053545	0.654656
	32	0.000761	7.3482	1.449350	0.045292	0.679963
	64	0.001298	4.3073	0.849564	0.013274	1.179886
1024	1	0.065843	5.4359	1.000000	1.000000	–
	2	0.036647	9.7665	1.796664	0.898332	0.113174
	4	0.021720	16.4786	3.031456	0.757864	0.106499
	6	0.016517	21.6693	3.986335	0.664389	0.101028
	8	0.014290	25.0460	4.607539	0.575942	0.105184
	10	0.013045	27.4374	5.047463	0.504746	0.109021
	12	0.012327	29.0339	5.341152	0.445096	0.113337
	14	0.011641	30.7451	5.655961	0.403997	0.113482
	16	0.011049	32.3938	5.959259	0.372454	0.112327
	18	0.010851	32.9844	6.067893	0.337105	0.115673
	20	0.010679	33.5163	6.165757	0.308288	0.118091
	24	0.010776	33.2154	6.110399	0.254600	0.127293
	28	0.010966	32.6386	6.004285	0.214439	0.135679
	32	0.011281	31.7280	5.836761	0.182399	0.144597
	64	0.014003	25.5599	4.702071	0.073470	0.200175

n	Threads	Runtime (s)	GFLOP/s	Speedup	Efficiency	Karp–Flatt ε
2048	1	0.522825	5.4766	1.000000	1.000000	–
	2	0.271680	10.5393	1.924413	0.962207	0.039278
	4	0.147000	19.4783	3.556628	0.889157	0.041554
	6	0.105858	27.0486	4.938919	0.823153	0.042968
	8	0.083343	34.3559	6.273187	0.784148	0.039324
	10	0.070763	40.4633	7.388367	0.738837	0.039275
	12	0.061982	46.1962	8.435170	0.702931	0.038420
	14	0.055723	51.3847	9.382552	0.670182	0.037856
	16	0.051425	55.6792	10.166712	0.635420	0.038251
	18	0.048506	59.0300	10.778545	0.598808	0.039411
	20	0.046612	61.4283	11.216467	0.560823	0.041215
	24	0.043301	66.1260	12.074230	0.503093	0.042944
	28	0.041721	68.6302	12.531495	0.447553	0.045717
	32	0.041079	69.7029	12.727350	0.397730	0.048847
	64	0.045879	62.4101	11.395738	0.178058	0.073272

Table 21: Extended thread-scaling metrics for v4 Final Version at $n \in \{256, 1024, 2048\}$.

.3 Code evolution across milestone versions

.3.1 v0: Baseline serial kernel

Tag: cholesky_v0_baseline.

```

1  #include "cholesky.h"
2
3  #include <chrono>
4  #include <cmath>
5  #include <cstdint>
6
7  double cholesky(double* c, int n) {
8      if (c == nullptr || n < 0 || n > 100000) {
9          return -1.0;
10     }
11
12     const std::size_t stride = static_cast<std::size_t>(n);
13     const auto start = std::chrono::steady_clock::now();
14
15     for (int p = 0; p < n; ++p) {
16         const std::size_t p_index = static_cast<std::size_t>(p);
17         const std::size_t diag_offset = p_index * stride + p_index;
18         const double diag_entry = c[diag_offset];
19
20         if (diag_entry <= 0.0) {
21             return -1.0;
22         }
23
24         const double diag = std::sqrt(diag_entry);
25         c[diag_offset] = diag;
26
27         for (int j = p + 1; j < n; ++j) {
28             const std::size_t j_offset = static_cast<std::size_t>(j);
29             c[p_index * stride + j_offset] /= diag;
30         }
31     }

```

```

32     for (int i = p + 1; i < n; ++i) {
33         const std::size_t i_offset = static_cast<std::size_t>(i);
34         c[i_offset * stride + p_index] /= diag;
35     }
36
37     for (int j = p + 1; j < n; ++j) {
38         const std::size_t j_offset = static_cast<std::size_t>(j);
39         const double row_value = c[p_index * stride + j_offset];
40
41         for (int i = p + 1; i < n; ++i) {
42             const std::size_t i_offset = static_cast<std::size_t>(i);
43             c[i_offset * stride + j_offset] -= c[i_offset * stride + p_index] * row_value;
44         }
45     }
46 }
47
48 const auto end = std::chrono::steady_clock::now();
49 const std::chrono::duration<double> elapsed = end - start;
50 return elapsed.count();
51 }

```

.3.2 v1: Serial kernel clean-up

Tag: cholesky_v1_serial_opt.

```

1  #include "cholesky.h"
2
3  #include <chrono>
4  #include <cmath>
5  #include <cstdint>
6
7  double cholesky(double* c, int n) {
8      if (c == nullptr || n < 0 || n > 100000) {
9          return -1.0;
10     }
11
12     const std::size_t stride = static_cast<std::size_t>(n);
13     const auto start = std::chrono::steady_clock::now();
14
15     for (int p = 0; p < n; ++p) {
16         const std::size_t p_index = static_cast<std::size_t>(p);
17         const std::size_t diag_offset = p_index * stride + p_index;
18         double* const row_p = c + p_index * stride;
19         const double diag_entry = c[diag_offset];
20
21         if (diag_entry <= 0.0) {
22             return -1.0;
23         }
24
25         const double diag = std::sqrt(diag_entry);
26         const double inv_diag = 1.0 / diag;
27         c[diag_offset] = diag;
28
29         for (int j = p + 1; j < n; ++j) {
30             const std::size_t j_offset = static_cast<std::size_t>(j);
31             row_p[j_offset] *= inv_diag;
32         }
33
34         for (int i = p + 1; i < n; ++i) {
35             const std::size_t i_offset = static_cast<std::size_t>(i);
36             c[i_offset * stride + p_index] *= inv_diag;
37         }
38
39         for (int i = p + 1; i < n; ++i) {
40             const std::size_t i_offset = static_cast<std::size_t>(i);
41             double* const row_i = c + i_offset * stride;
42             const double col_value = row_i[p_index];
43
44             for (int j = p + 1; j < n; ++j) {

```



```

45         const std::size_t j_offset = static_cast<std::size_t>(j);
46         row_i[j_offset] -= col_value * row_p[j_offset];
47     }
48 }
49 }
50
51 const auto end = std::chrono::steady_clock::now();
52 const std::chrono::duration<double> elapsed = end - start;
53 return elapsed.count();
54 }

```

.3.3 v2: Blocked serial formulation

Tag: cholesky_v2_serial_blocked.

```

1  #include "cholesky.h"
2
3  #include <chrono>
4  #include <cmath>
5  #include <cstdint>
6  #include <algorithm>
7  #include <cstdlib>
8
9  namespace {
10
11  int resolve_block_size() {
12      constexpr int default_block_size = 32;
13      const char* value = std::getenv("CHOLESKY_BLOCK_SIZE");
14      if (value == nullptr) {
15          return default_block_size;
16      }
17
18      const int parsed = std::atoi(value);
19      return parsed > 0 ? parsed : default_block_size;
20  }
21
22  } // namespace
23
24  double cholesky(double* c, int n) {
25      if (c == nullptr || n < 0 || n > 100000) {
26          return -1.0;
27      }
28
29      const std::size_t stride = static_cast<std::size_t>(n);
30      const auto start = std::chrono::steady_clock::now();
31      const int panel_block_size = resolve_block_size();
32      const int trailing_block_size = panel_block_size;
33
34      auto at = [c, stride](int row, int col) -> double& {
35          return c[static_cast<std::size_t>(row) * stride + static_cast<std::size_t>(col)];
36      };
37
38      for (int block_start = 0; block_start < n; block_start += panel_block_size) {
39          const int block_end = std::min(block_start + panel_block_size, n);
40
41          // Factor the diagonal panel one column at a time while keeping it mirrored.
42          for (int p = block_start; p < block_end; ++p) {
43              const double diag_entry = at(p, p);
44              if (diag_entry <= 0.0) {
45                  return -1.0;
46              }
47
48              const double diag = std::sqrt(diag_entry);
49              const double inv_diag = 1.0 / diag;
50              at(p, p) = diag;
51
52              for (int i = p + 1; i < block_end; ++i) {
53                  const double value = at(i, p) * inv_diag;
54                  at(i, p) = value;

```

```

55         at(p, i) = value;
56     }
57
58     for (int j = p + 1; j < block_end; ++j) {
59         const double panel_value = at(j, p);
60         for (int i = j; i < block_end; ++i) {
61             const double updated = at(i, j) - at(i, p) * panel_value;
62             at(i, j) = updated;
63             if (i != j) {
64                 at(j, i) = updated;
65             }
66         }
67     }
68 }
69
70 // Solve the block column below the panel using the newly factorized block.
71 for (int i = block_end; i < n; ++i) {
72     for (int j = block_start; j < block_end; ++j) {
73         double value = at(i, j);
74         for (int p = block_start; p < j; ++p) {
75             value -= at(i, p) * at(j, p);
76         }
77         value /= at(j, j);
78         at(i, j) = value;
79         at(j, i) = value;
80     }
81 }
82
83 // Update the trailing matrix with blocked rank-k style tiles.
84 for (int i_block = block_end; i_block < n; i_block += trailing_block_size) {
85     const int i_end = std::min(i_block + trailing_block_size, n);
86
87     for (int j_block = block_end; j_block <= i_block; j_block += trailing_block_size) {
88         const int j_end = std::min(j_block + trailing_block_size, n);
89
90         for (int i = i_block; i < i_end; ++i) {
91             const int j_limit = std::min(j_end, i + 1);
92             for (int j = j_block; j < j_limit; ++j) {
93                 double value = at(i, j);
94                 for (int p = block_start; p < block_end; ++p) {
95                     value -= at(i, p) * at(j, p);
96                 }
97                 at(i, j) = value;
98                 if (i != j) {
99                     at(j, i) = value;
100                 }
101             }
102         }
103     }
104 }
105 }
106
107 const auto end = std::chrono::steady_clock::now();
108 const std::chrono::duration<double> elapsed = end - start;
109 return elapsed.count();
110 }

```

.3.4 v3: First OpenMP version

Tag: cholesky_v3_openmp.

```

1  #include "cholesky.h"
2
3  #include <chrono>
4  #include <cmath>
5  #include <cstdint>
6  #include <algorithm>
7  #include <cstdlib>
8  #include <omp.h> // OpenMP is used for the first parallel version in the trailing-update hotspot.

```

```

9
10 namespace {
11
12 int resolve_block_size() {
13     constexpr int default_block_size = 32;
14     const char* value = std::getenv("CHOLESKY_BLOCK_SIZE");
15     if (value == nullptr) {
16         return default_block_size;
17     }
18
19     const int parsed = std::atoi(value);
20     return parsed > 0 ? parsed : default_block_size;
21 }
22
23 } // namespace
24
25 double cholesky(double* c, int n) {
26     if (c == nullptr || n < 0 || n > 100000) {
27         return -1.0;
28     }
29
30     const std::size_t stride = static_cast<std::size_t>(n);
31     const auto start = std::chrono::steady_clock::now();
32     const int panel_block_size = resolve_block_size();
33     const int trailing_block_size = panel_block_size;
34
35     auto at = [c, stride](int row, int col) -> double& {
36         return c[static_cast<std::size_t>(row) * stride + static_cast<std::size_t>(col)];
37     };
38
39     for (int block_start = 0; block_start < n; block_start += panel_block_size) {
40         const int block_end = std::min(block_start + panel_block_size, n);
41
42         // Factor the diagonal panel one column at a time while keeping it mirrored.
43         for (int p = block_start; p < block_end; ++p) {
44             const double diag_entry = at(p, p);
45             if (diag_entry <= 0.0) {
46                 return -1.0;
47             }
48
49             const double diag = std::sqrt(diag_entry);
50             const double inv_diag = 1.0 / diag;
51             at(p, p) = diag;
52
53             for (int i = p + 1; i < block_end; ++i) {
54                 const double value = at(i, p) * inv_diag;
55                 at(i, p) = value;
56                 at(p, i) = value;
57             }
58
59             for (int j = p + 1; j < block_end; ++j) {
60                 const double panel_value = at(j, p);
61                 for (int i = j; i < block_end; ++i) {
62                     const double updated = at(i, j) - at(i, p) * panel_value;
63                     at(i, j) = updated;
64                     if (i != j) {
65                         at(j, i) = updated;
66                     }
67                 }
68             }
69         }
70
71         // Solve the block column below the panel using the newly factorized block.
72         for (int i = block_end; i < n; ++i) {
73             for (int j = block_start; j < block_end; ++j) {
74                 double value = at(i, j);
75                 for (int p = block_start; p < j; ++p) {
76                     value -= at(i, p) * at(j, p);
77                 }
78                 value /= at(j, j);
79                 at(i, j) = value;
80                 at(j, i) = value;
81             }

```

```

82     }
83
84     // Update the trailing matrix with blocked rank-k style tiles.
85     // OpenMP is introduced here first because these trailing tiles are independent for a fixed panel
    and dominate the runtime.
86     #pragma omp parallel for schedule(static) if (n - block_end > trailing_block_size)
87     for (int i_block = block_end; i_block < n; i_block += trailing_block_size) {
88         const int i_end = std::min(i_block + trailing_block_size, n);
89
90         for (int j_block = block_end; j_block <= i_block; j_block += trailing_block_size) {
91             const int j_end = std::min(j_block + trailing_block_size, n);
92
93             for (int i = i_block; i < i_end; ++i) {
94                 const int j_limit = std::min(j_end, i + 1);
95                 for (int j = j_block; j < j_limit; ++j) {
96                     double value = at(i, j);
97                     for (int p = block_start; p < block_end; ++p) {
98                         value -= at(i, p) * at(j, p);
99                     }
100                     at(i, j) = value;
101                     if (i != j) {
102                         at(j, i) = value;
103                     }
104                 }
105             }
106         }
107     }
108 }
109
110 const auto end = std::chrono::steady_clock::now();
111 const std::chrono::duration<double> elapsed = end - start;
112 return elapsed.count();
113 }

```

.3.5 v4: Final tuned OpenMP kernel

Tag: cholesky_v4_parallel_opt.

```

1  #include "cholesky.h"
2
3  #include <algorithm>
4  #include <chrono>
5  #include <cmath>
6  #include <cstdint>
7  #include <cstdlib>
8  #include <omp.h> // OpenMP is used for the first parallel version in the trailing-update hotspot.
9  #include <string>
10
11 namespace {
12
13 int resolve_positive_env_or_default(const char* name, int default_value) {
14     const char* value = std::getenv(name);
15     if (value == nullptr) {
16         return default_value;
17     }
18
19     const int parsed = std::atoi(value);
20     return parsed > 0 ? parsed : default_value;
21 }
22
23 int resolve_block_size() {
24     return resolve_positive_env_or_default("CHOLESKY_BLOCK_SIZE", 32);
25 }
26
27 //This helper stores the runtime OpenMP schedule policy and chunk size used by the tuned final kernel.
28 struct OpenMPScheduleConfig {
29     //schedule is kept explicit for benchmarking workflow can compare static, dynamic, and guided policies.
30     omp_sched_t kind = omp_sched_guided;
31     int chunk = 1;

```

```

32 };
33
34 OpenMPScheduleConfig resolve_openmp_schedule() {
35     OpenMPScheduleConfig config;
36
37     if (const char* schedule_value = std::getenv("CHOLESKY_OMP_SCHEDULE")) {
38         const std::string schedule(schedule_value);
39         if (schedule == "dynamic") {
40             config.kind = omp_sched_dynamic;
41         } else if (schedule == "guided") {
42             config.kind = omp_sched_guided;
43         } else if (schedule == "auto") {
44             config.kind = omp_sched_auto;
45         } else {
46             config.kind = omp_sched_static;
47         }
48     }
49
50     if (const char* chunk_value = std::getenv("CHOLESKY_OMP_CHUNK")) {
51         const int parsed_chunk = std::atoi(chunk_value);
52         if (parsed_chunk > 0) {
53             config.chunk = parsed_chunk;
54         }
55     }
56
57     return config;
58 }
59
60 } // namespace
61
62 double cholesky(double* c, int n) {
63     if (c == nullptr || n < 0 || n > 100000) {
64         return -1.0;
65     }
66
67     const std::size_t stride = static_cast<std::size_t>(n);
68     const int block_size = resolve_block_size();
69     const OpenMPScheduleConfig openmp_schedule = resolve_openmp_schedule();
70
71     // Program the OpenMP runtime from the tuned default schedule, while still allowing the v4 workflow
72     // to override policy through the environment.
73     omp_set_schedule(openmp_schedule.kind, openmp_schedule.chunk);
74
75     auto row_ptr = [c, stride](int row) -> double* {
76         return c + static_cast<std::size_t>(row) * stride;
77     };
78
79     const auto start = std::chrono::steady_clock::now();
80
81     for (int block_start = 0; block_start < n; block_start += block_size) {
82         const int block_end = std::min(block_start + block_size, n);
83
84         // Factor the diagonal panel with an unblocked lower-triangular kernel while preserving the
85         // mirrored upper triangle required by the coursework.
86         for (int p = block_start; p < block_end; ++p) {
87             double* const row_p = row_ptr(p);
88             const double diag_entry = row_p[p];
89             if (diag_entry <= 0.0) {
90                 return -1.0;
91             }
92
93             const double diag = std::sqrt(diag_entry);
94             const double inv_diag = 1.0 / diag;
95             row_p[p] = diag;
96
97             for (int i = p + 1; i < block_end; ++i) {
98                 double* const row_i = row_ptr(i);
99                 row_i[p] *= inv_diag;
100                 row_p[i] = row_i[p];
101             }
102
103             for (int j = p + 1; j < block_end; ++j) {
104                 double* const row_j = row_ptr(j);

```

```

103         const double panel_value = row_j[p];
104
105         for (int i = j; i < block_end; ++i) {
106             double* const row_i = row_ptr(i);
107             const double updated = row_i[j] - row_i[p] * panel_value;
108             row_i[j] = updated;
109             if (i != j) {
110                 row_j[i] = updated;
111             }
112         }
113     }
114 }
115
116 // Solve the block column below the panel after panel factorization; rows below the panel are
117 // independent, so this is a safe OpenMP region.
118 // This `parallel for` distributes independent rows of the solved block column to reduce the
119 // panel-to-trailing handoff cost.
120 #pragma omp parallel for schedule(runtime) if (n - block_end > block_size)
121 for (int i = block_end; i < n; ++i) {
122     double* const row_i = row_ptr(i);
123
124     for (int j = block_start; j < block_end; ++j) {
125         double* const row_j = row_ptr(j);
126         double dot = 0.0;
127
128         // This `simd` reduction vectorizes the panel dot product because each term is
129         // independent and feeds one scalar accumulation.
130         #pragma omp simd reduction(+:dot)
131         for (int p = block_start; p < j; ++p) {
132             dot += row_i[p] * row_j[p];
133         }
134
135         row_i[j] = (row_i[j] - dot) / row_j[j];
136         row_j[i] = row_i[j];
137     }
138 }
139
140 // Update the trailing matrix tile by tile; each outer i-block owns disjoint row ranges in the
141 // lower triangle, making it the main OpenMP hotspot.
142 // This `parallel for` spreads independent trailing tiles across threads while preserving
143 // deterministic writes to disjoint lower-triangular regions.
144 #pragma omp parallel for schedule(runtime) if (n - block_end > block_size)
145 for (int i_block = block_end; i_block < n; i_block += block_size) {
146     const int i_end = std::min(i_block + block_size, n);
147
148     for (int j_block = block_end; j_block <= i_block; j_block += block_size) {
149         const int j_end = std::min(j_block + block_size, n);
150
151         for (int i = i_block; i < i_end; ++i) {
152             double* const row_i = row_ptr(i);
153             const int j_limit = std::min(j_end, i + 1);
154
155             for (int j = j_block; j < j_limit; ++j) {
156                 double* const row_j = row_ptr(j);
157                 double dot = 0.0;
158
159                 // This `simd` reduction vectorizes the rank-k accumulation inside each tile
160                 // because the dot-product terms are independent.
161                 #pragma omp simd reduction(+:dot)
162                 for (int p = block_start; p < block_end; ++p) {
163                     dot += row_i[p] * row_j[p];
164                 }
165
166                 row_i[j] -= dot;
167                 if (i != j) {
168                     row_j[i] = row_i[j];
169                 }
170             }
171         }
172     }
173 }

```

```
170     const auto end = std::chrono::steady_clock::now();
171     const std::chrono::duration<double> elapsed = end - start;
172     return elapsed.count();
173 }
```

Bibliography

- [1] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2 edition, 2002.
- [2] Jack J. Dongarra, Jeremy Du Croz, Sven Hammarling, and Iain S. Duff. A set of level 3 basic linear algebra subprograms. *ACM Transactions on Mathematical Software*, 16(1):1–17, 1990.
- [3] E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen. *LAPACK Users' Guide*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 3 edition, 1999.
- [4] James W. Demmel and Nicholas J. Higham. Stability of block algorithms with fast level-3 BLAS. *ACM Transactions on Mathematical Software*, 18(3):274–291, 1992.
- [5] OpenMP Architecture Review Board. OpenMP application programming interface version 5.2, 2021. Accessed 2026-03-31.
- [6] Alan H. Karp and Horace P. Flatt. Measuring parallel processor performance. *Communications of the ACM*, 33(5):539–543, 1990.
- [7] NumPy Developers. `numpy.linalg.slogdet`, 2024. NumPy Reference Manual, accessed 2026-03-31.